

PRACTICAL 6

Considered there are N philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. one chopstick may be picked up by any one of its adjacent followers but not both.

Write a program to solve the problem using process synchronization

Code:

```
main.py
1 import threading
2 import time
3
4 # Number of philosophers
5 N = 5
6
7 # Mutexes for chopsticks
8 chopsticks = [threading.Lock() for _ in range
9               (N)]
10
11 # Philosopher class
12 class Philosopher(threading.Thread):
13     def __init__(self, index):
14         super().__init__()
15         self.index = index
16
17     def run(self):
18         while True:
19             self.think() # Philosopher thinks
20                         # for a while
21             self.eat()   # Philosopher tries
```

```
main.py
17 while True:
18     self.think() # Philosopher thinks
19                 # for a while
20     self.eat()   # Philosopher tries
21                 # to eat
22
23     def think(self):
24         print(f"Philosopher {self.index} is
25               thinking.")
26         time.sleep(2) # Think for 2 seconds
27
28     def eat(self):
29         # Philosopher tries to pick up both
30         # chopsticks
31         left_chopstick = self.index
32         right_chopstick = (self.index + 1) % N
33
34         # To avoid deadlock, philosophers pick
35         # chopsticks in order (lower index
36         # first)
```

```
main.py  [Icons] [Run]

31     first, second = min(left_chopstick,
32                           right_chopstick), max
33                           (left_chopstick, right_chopstick)
34     # Try to pick up both chopsticks
35     with chopsticks[first]:
36         print(f"Philosopher {self.index}
37             picked up chopstick {first}")
38     with chopsticks[second]:
39         print(f"Philosopher {self
40             .index} picked up
41             chopstick {second}")
42     self.eating()
43
44 def eating(self):
45     # The philosopher eats for a while
46     print(f"Philosopher {self.index} is
47         eating.")
48     time.sleep(3) # Eating for 3 seconds
49     print(f"Philosopher {self.index}
```

```
main.py  [Icons] [Run]

43     time.sleep(3) # Eating for 3 seconds
44     print(f"Philosopher {self.index}
45         finished eating.")
46
47 # Main function to simulate the dining
48 philosophers
49
50 def main():
51     philosophers = []
52
53     # Create philosopher threads
54     for i in range(N):
55         philosopher = Philosopher(i)
56         philosophers.append(philosopher)
57
58     # Start philosopher threads
59     for philosopher in philosophers:
60         philosopher.start()
61
62     # Join philosopher threads (we don't
63     actually stop them in this simple
64     example)
```

```
54
55     # Start philosopher threads
56     for philosopher in philosophers:
57         philosopher.start()
58
59     # Join philosopher threads (we don't
60     actually stop them in this simple
61     example)
62     for philosopher in philosophers:
63         philosopher.join()
64
65 # Run the simulation
66 if __name__ == "__main__":
67     main()
```

Output :

```
Output Clear
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 picked up chopstick 0
Philosopher 0 picked up chopstick 1
Philosopher 0 is eating.
Philosopher 2 picked up chopstick 2
Philosopher 2 picked up chopstick 3
Philosopher 2 is eating.
Philosopher 0 finished eating.
Philosopher 0 is thinking.
Philosopher 1 picked up chopstick 1
Philosopher 4 picked up chopstick 0
Philosopher 4 picked up chopstick 4
Philosopher 4 is eating.
Philosopher 2 finished eating.
Philosopher 2 is thinking.
Philosopher 3 picked up chopstick 3
Philosopher 1 picked up chopstick 2
```